



Licenciatura Engenharia Informática e Multimédia

Instituto Superior de Engenharia de Lisboa

Ano letivo 2024/2025

Produção e Conteúdos Multimédia

Docente: Pedro Albuquerque Santos

Projeto

Grupo: Q6

Turma: 32 D

Nome: Pedro Marques

Número: 51959

Nome: Gianni Floriddia

Número: 51945

Data: 28 de Dezembro de 2024

Conteúdo

Lista de Figuras.....	3
1. Introdução	4
2. Descrição das Funcionalidades	4
2.1. Pesquisa por Palavras-Chave	4
2.2. Pesquisa por Cores	4
2.3. Nuvem de Palavras	4
2.4. Integração de Áudio	5
2.5. Animações na Interface	5
2.6. Testes de Usabilidade	5
3. Organização da Aplicação	5
4. Código	6
4.1. Classe ColorHistogram	6
4.2. Classe ISearchEngine	8
4.3. Classe ISearchEngine_Manager	13
5. Interface	16
6. Extra - Questionário.....	17
7. Conclusões	18

Lista de Figuras

Figura 1: Função countPixels(pixels){}	7
Figura 2: Função databaseProcessing(cnv){}	9
Figura 3: Função createColorbaseLS(){}	10
Figura 4: Função searchColor(category,color){}	11
Figura 5: Função gridView(canvas){}	12
Figura 6: Aparência do projeto	16
Figura 7: Aparência de uma página do questionário	17

1. Introdução

No âmbito da unidade curricular de Produção de Conteúdos Multimédia, foi desenvolvido o projeto final que tem como objetivo a criação de uma aplicação interativa intitulada "Image Gallery". Este projeto visa aplicar e consolidar os conhecimentos adquiridos ao longo do semestre nas áreas de HTML5, CSS e JavaScript, promovendo a implementação de soluções multimédia inovadoras centradas no utilizador.

A "Image Gallery" consiste numa galeria de imagens dinâmica e funcional, com funcionalidades que permitem a pesquisa e visualização de imagens com base em palavras-chave e cores.

Para além dos requisitos mínimos, foram implementadas funcionalidades adicionais que enriquecem a experiência de utilização, como animações, integração de áudio e utilização de algoritmos para processamento de imagens.

Este relatório documenta todo o processo de desenvolvimento da aplicação, desde a sua conceção até à implementação e avaliação. Além de descrever as soluções implementadas, serão discutidos os principais desafios enfrentados e as respetivas soluções encontradas, bem como os resultados obtidos durante os testes de utilização.

2. Descrição das Funcionalidades

A aplicação foi projetada para oferecer uma experiência rica e interativa, incorporando funcionalidades que vão além dos requisitos mínimos definidos para o projeto. Estas funcionalidades incluem:

2.1. Pesquisa por Palavras-Chave

A aplicação permite ao utilizador realizar pesquisas utilizando palavras-chave. Para facilitar o processo, foi implementada uma nuvem de palavras no ecrã inicial, onde o utilizador pode clicar para selecionar as palavras-chave pretendidas. Esta funcionalidade visa otimizar a experiência do utilizador, reduzindo o esforço de introdução manual e aumentando a rapidez na busca de resultados.

2.2. Pesquisa por Cores

Uma funcionalidade adicional é a pesquisa baseada no histograma. O histograma apresenta uma paleta de 12 cores principais, permitindo ao utilizador clicar na cor desejada e visualizar imagens com tonalidades semelhantes. Esta abordagem oferece uma forma inovadora de interação e navegação visual pela galeria.

2.3. Nuvem de Palavras

Foi implementada uma nuvem de palavras onde o utilizador poderá simplesmente escolher a categoria das fotos clicando no nome da própria.

2.4. Integração de Áudio

A aplicação incorpora música de fundo para melhorar a experiência do utilizador. Este elemento sonoro pode ser controlado através de um botão que permite iniciar, pausar ou retomar a música. Esta funcionalidade contribui para criar um ambiente mais imersivo e agradável durante a interação com a galeria. Para além disso também foram incorporados efeitos sonoros nos botões.

2.5. Animações na Interface

Para melhorar a usabilidade e tornar a interface mais intuitiva, foram introduzidas animações suaves em diferentes elementos da aplicação. Por exemplo, animações incorporadas nos botões, pesquisa, seleção de cores e no canvas. Estas animações oferecem feedback visual ao utilizador, tornando as interações mais claras.

2.6. Testes de Usabilidade

Foi conduzida uma avaliação da aplicação com base em testes de utilizador, envolvendo pelo menos cinco participantes. Estes testes avaliaram aspetos como a facilidade de navegação, clareza das funcionalidades e satisfação geral.

3. Organização da Aplicação

A aplicação é composta por uma página principal em HTML e uma folha de estilos em CSS. Adicionalmente, foram desenvolvidos vários ficheiros JavaScript, cada um com responsabilidades específicas:

- **Database.js:** Responsável por interagir com o LocalStorage, garantindo o armazenamento e recuperação de dados persistentes.
- **Image_Processing.js:** Utilizado para o processamento de imagens, incluindo a geração e manipulação de histogramas.
- **ISearchEngine.js:** Contém as funções relacionadas ao processamento de imagens, manipulação do elemento *canvas* e criação da base de dados associada.
- **ISearchEngine_Manager.js:** É o núcleo da aplicação, responsável por integrar o código com o HTML, gerir a inicialização da aplicação e assegurar o funcionamento geral da solução.

Com esta arquitetura modular, a aplicação está organizada de forma a facilitar a manutenção e a evolução futura.

4. Código

4.1. Classe ColorHistogram

A classe ColorHistogram é projetada para processar e analisar dados de cores em imagens. Ela permite que os valores de cada canal de cor (vermelho, verde e azul) sejam organizados e analisados para criar histogramas baseados na proximidade das cores. Esses histogramas representam a quantidade de pixels que pertencem a cada uma das categorias de cores definidas.

Funcionamento Geral

- **Inicialização:** No construtor da classe, os dados de entrada consistem em três arrays correspondentes aos valores dos canais de cor RGB (vermelho, verde e azul). Estes valores são combinados para criar um array de tripletos [R, G, B], onde cada triplo representa uma categoria de cor.
- **Histograma:** A função principal da classe é calcular um histograma que contabiliza a frequência de pixels pertencentes a cada uma das categorias de cor definidas. A contagem baseia-se na distância entre as cores de cada pixel e os valores RGB definidos no array `this.colors`.

Explicação da Função `countPixels`

A função `countPixels` realiza a análise e classificação de cada pixel da imagem em uma das categorias de cor definidas.

Funcionamento

1. **Parâmetro de Entrada:**
 - A função recebe um objeto `ImageData` como argumento. Este objeto contém os dados da imagem, incluindo os valores de cada canal de cor (vermelho, verde e azul) e um canal de opacidade (alfa), organizados em um array unidimensional.
2. **Inicialização do Histograma:**
 - Um array chamado `hist` é criado com 12 posições (ou bins), todas inicializadas em zero. Este array representa a contagem de pixels para cada categoria de cor.
3. **Iteração pelos Pixels:**
 - A função percorre o array `data` da imagem em incrementos de 4 (um para cada canal: R, G, B e Alfa). Para cada pixel, os valores RGB são extraídos.
4. **Cálculo da Cor Mais Próxima:**
 - Para cada pixel, a função calcula a distância entre os valores RGB do pixel e os valores RGB de todas as categorias de cores definidas em `this.colors`.

A distância é calculada como a soma das diferenças absolutas entre os valores dos canais de cor ($|R1 - R2| + |G1 - G2| + |B1 - B2|$).

- O índice da cor mais próxima (menor distância) é identificado e armazenado.

5. Incremento no Histograma:

- Após encontrar a cor mais próxima, a função incrementa o valor correspondente no array hist para registrar a contagem do pixel naquela categoria.

6. Resultado:

- No final, a função retorna o array hist, que contém a quantidade de pixels pertencentes a cada categoria de cor.

```
/**
 * Counts the number of pixels corresponding to defined color categories.
 * @param {ImageData} pixels - Image data containing the color information for the image.
 * @returns {number[]} - Histogram data representing pixel counts for each color category.
 */
countPixels(pixels : ImageData) : number[] { Show usages  ⓘ Gianni
  const hist : any[] = new Array( arrayLength: 12 ).fill( value: 0 ); // Initialize histogram with 12 bins, set to zero
  const data : Uint8ClampedArray = pixels.data;

  for (let i : number = 0; i < data.length; i += 4) {
    const r : number = data[i];
    const g = data[i + 1];
    const b = data[i + 2];

    let closestIndex : number = 0;
    let minDistance : number = Infinity;

    for (let j : number = 0; j < this.colors.length; j++) {
      const [cr, cg, cb] = this.colors[j];
      const distance : number = Math.abs( x: r - cr ) + Math.abs( x: g - cg ) + Math.abs( x: b - cb );

      if (distance < minDistance) {
        minDistance = distance;
        closestIndex = j;
      }
    }

    hist[closestIndex]++;
  }

  return hist; // Return the histogram data
}
```

Figura 1: Função countPixels(pixels){}

4.2. Classe ISearchEngine

A classe ISearchEngine implementa um mecanismo de busca de imagens baseado em cor e palavras-chave. A ideia é carregar um banco de dados JSON contendo informações das imagens, processá-las com base em histogramas de cor, e fornecer funcionalidades para pesquisar e exibir imagens.

Construtor

- Inicializa os atributos da classe, criando instâncias para manipulação do banco de dados e preparando os arrays de cores e categorias.
-

1. init(cnv)

1. Define o canvas de visualização (this.viewCanvas).
 2. Carrega os dados do banco JSON usando o método loadFile da classe DatabaseJSON.
 3. Verifica se o Local Storage está vazio:
 - Caso esteja vazio, chama o método databaseProcessing para processar todas as imagens.
 - Caso contrário, utiliza os dados já processados no Local Storage.
 4. Adiciona um event listener ao canvas para lidar com o efeito "hover", chamando o método handleHover quando o mouse se move sobre o canvas.
-

2. databaseProcessing(cnv)

Objetivo: Processar todas as imagens do banco de dados, calculando histogramas de cor e outras propriedades.

1. Instancia a classe ColorHistogram para realizar a análise de cores.
2. Itera sobre cada imagem do banco de dados JSON:
 - Cria um evento exclusivo para cada imagem processada (ex.: processed_picture_<image_path>).
 - Adiciona um event listener que chama imageProcessed assim que o processamento da imagem é concluído.
3. Para cada imagem, chama o método computation (provavelmente da classe Picture) para realizar o processamento com o canvas invisível (processingCanvas).

```

/**
 * Processes the image database for color histogram and color moments.
 * @param {HTMLCanvasElement} cnv - The canvas element for image processing.
 */
databaseProcessing(cnv):void { Show usages  Gianni
  // Image processing classes
  const h12color : ColorHistogram = new ColorHistogram(this.redColor, this.greenColor, this.blueColor); // Histogram processing

  for (const imageData of this.jsonData.images) {
    let img : Picture = new Picture( px: 0, py: 0, w: 100, h: 100, imageData.path, imageData.class);

    // Create an event name for when the image is processed
    const eventname : string = "processed_picture_" + img.ipath;
    const eventP : Event = new Event(eventname);

    // Listen for the "processed_picture_" event and call imageProcessed when it's fired
    document.addEventListener(eventname, listener: () :void => {
      this.imageProcessed(img, eventname);
    });

    // Start processing the image with the provided canvas and processing classes
    img.computation(cnv, h12color, eventP);
  }
}

```

Figura 2: Função `databaseProcessing(cnv){}`

3. imageProcessed(img, eventname)

Objetivo: Atualizar o estado do mecanismo de busca quando uma imagem é processada.

1. Verifica se o Local Storage está vazio:
 - Se estiver, insere a imagem no pool de imagens processadas (`this.allpictures`).
 2. Exibe o progresso do processamento na interface (número de imagens processadas até o momento).
 3. Quando todas as imagens são processadas:
 - Salva as informações no Local Storage chamando o método `createColorDatabaseLS`.
-

4. createColorDatabaseLS()

Objetivo: Criar uma base de dados no Local Storage para facilitar consultas baseadas em cores.

1. Agrupa as imagens processadas por categoria.
2. Para cada categoria:
 - Ordena as imagens pelo número de pixels da cor dominante usando `sortByColor`.

- Seleciona até `this.numShownPic` imagens para salvar.

3. Salva os dados no Local Storage, organizados por categorias e cores.

```
/**
 * Creates the color database in Local Storage.
 * This method creates the color query database in localStorage by saving
 * the images grouped or indexed by their dominant colors.
 */
createColorDatabaseLS() : void { Show usages  ⓘ Gianni
  console.log("Create Color Database");
  const imagesByCategory : {} = {};
  this.allPictures.stuff.forEach(picture => {
    if (!imagesByCategory[picture.category])
      imagesByCategory[picture.category] = [];
    imagesByCategory[picture.category].push(picture);
  });
  console.log("Images by Category", imagesByCategory);

  const colorDatabase : {} = {};
  this.categories.forEach(category : string => {
    colorDatabase[category] = {images: []};
    this.colors.forEach((color : string, i : number) : void => {
      console.log(imagesByCategory[category]);
      this.sortbyColor(i, imagesByCategory[category]);
      colorDatabase[category].images.push(
        ...imagesByCategory[category].slice(0, this.numShownPic).map(img => ({class: color, image: img}))
      )
    })
  })

  for (const category in colorDatabase) {
    this.localStorage.save(category, colorDatabase[category]);
  }
  console.log("Color Database", colorDatabase);
}
```

Figura 3: Função `createColorbaseLS()`

4. Exemplo de estrutura salva:

```
{
  "burj khalifa": {
    "images": [
      { "class": "red", "image": { "impath": "path/to/image1.jpg", ... } },
      { "class": "blue", "image": { "impath": "path/to/image2.jpg", ... } }
    ]
  },
  ...
}
```

5. `searchColor(category, color)`

Objetivo: Buscar imagens com base em uma cor e, opcionalmente, em uma categoria.

Funcionamento:

1. Se nenhuma categoria for especificada (category === ""):
 - Itera por todas as categorias no Local Storage.
 - Filtra as imagens com a cor especificada (im.class === color).
 - Adiciona até 3 imagens de cada categoria aos resultados.
2. Se uma categoria for especificada:
 - Busca diretamente nessa categoria no Local Storage.
 - Filtra as imagens com a cor especificada.
3. Atualiza o array this.results com os caminhos das imagens encontradas.
4. Chama o método gridView para renderizar as imagens no canvas.

```
/**
 * Searches images based on a selected color.
 * @param {string} category - The category to search within.
 * @param {string} color - The selected color.
 */
searchColor(category, color) :void { Show usages  ⓘ Gianni

    let results :any[] = [];

    if (category === "") {
        console.log("Searching by Color:", color);

        for (let i :number = 0; i < localStorage.length; i++) {
            const key :string = localStorage.key(i);
            const categoryData = this.localStorage.read(key);

            if (categoryData && categoryData.images) {
                const filteredImages :string[] = categoryData.images
                    .filter(im :T => im.class === color) // Filtra pela cor
                    .map(im :T => im.image.impath); // Extrai o caminho da imagem
                results.push(...filteredImages.slice(0, 3));
            }
        }
    } else {
        console.log("Searching by: ", category, " and Color:", color);
        results = this.localStorage.read(category).images.filter(im :T => im.class === color).map(im :T => im.image.impath);
    }

    this.results = results;
    console.log("RESULTADOS: ", this.results);
    this.gridView(this.viewCanvas);
}
```

Figura 4: Função searchColor(category,color){}

6. gridView(canvas)

Objetivo: Renderizar as imagens da pesquisa em um layout de grade no canvas.

1. Calcula o número de imagens que podem ser exibidas por linha, considerando a largura do canvas e o tamanho das imagens.
2. Posiciona cada imagem em uma grade com espaçamento horizontal e vertical.

3. Itera sobre os resultados (this.results) e renderiza cada imagem no canvas.

```
/**
 * Displays images in a grid view on a specified canvas.
 * @param {HTMLCanvasElement} canvas - The canvas element for rendering the grid view.
 * This method arranges the images in a grid on the canvas. The number of images per row
 * and the layout of the grid should be determined dynamically, depending on the size of the canvas.
 * It will draw the images in their corresponding positions, creating a visual grid of images.
 */
gridView(canvas) : void { Show usages  Gianni

    const ctx : CanvasRenderingContext2D = canvas.getContext( contextId: '2d');
    ctx.clearRect( x: 0, y: 0, canvas.width, canvas.height);
    const numImagesPerRow : number = Math.floor( x: canvas.width / this.imgWidth);
    const horizontalSpacing : number = (canvas.width - (numImagesPerRow * this.imgWidth)) / (numImagesPerRow + 1);
    const verticalSpacing : number = 10; // Example spacing between rows

    this.results.forEach((impath, index : number) : void => {
        const pictureElement : Picture = new Picture( px: index * 100, py: 0, w: 0, h: 100, impath, cat: "results");
        const row : number = Math.floor( x: index / numImagesPerRow);
        const col : number = index % numImagesPerRow;
        const x : number = col * this.imgWidth + horizontalSpacing * (col + 1);
        const y : number = row * (this.imgHeight + verticalSpacing);

        const imgElement : HTMLImageElement = new Image();
        imgElement.src = pictureElement.impath;
        imgElement.onload = () : void => {
            ctx.drawImage(imgElement, x, y, this.imgWidth, this.imgHeight);
        };
    })
}
```

Figura 5: Função gridView(canvas){}

7. handleHover(event)

Objetivo: Ampliar a imagem sob o cursor (efeito hover).

1. Calcula a posição do mouse relativa ao canvas.
2. Determina qual imagem está sob o cursor com base em suas coordenadas e tamanho.
3. Se o índice da imagem "hovered" mudou desde o último movimento do mouse:
 - Limpa o canvas.
 - Redesenha todas as imagens, ampliando apenas a imagem sob o cursor (em 10%).

8. sortByColor(idxColor, list)

Objetivo: Ordenar imagens por número de pixels de uma cor específica.

Funcionamento:

1. Acessa o histograma de cada imagem na lista (list).
2. Usa o índice da cor (idxColor) para comparar o número de pixels.
3. Ordena em ordem decrescente:
 - Imagens com mais pixels dessa cor aparecem primeiro.

Exemplo:

- list[0].hist[idxColor] = 150 (vermelho).
 - list[1].hist[idxColor] = 100.
 - A ordem será: [list[0], list[1]].
-

Conclusão:

As funções descritas acima são os pilares do funcionamento da classe ISearchEngine. Elas realizam desde o processamento e armazenamento das imagens até a busca e exibição de resultados, garantindo uma experiência interativa e eficiente para o usuário.

4.3. Classe ISearchEngine_Manager

Este ficheiro serve como o ponto de entrada principal da aplicação. Ele configura, inicializa e controla os elementos centrais do mecanismo de busca de imagens, integrando a lógica do motor de busca com a interface do utilizador.

Objetivo Geral

- **Inicialização da Aplicação:** Configura o canvas onde as imagens serão renderizadas, inicializa a classe ISearchEngine, e assegura que a aplicação é responsiva (ajustando o canvas com base no tamanho da janela).
 - **Interatividade do Usuário:** Adiciona event listeners para elementos da interface, como botões, caixas de cores, e botões de rádio, permitindo ao utilizador interagir com a aplicação.
 - **Integração com Áudio:** Reproduz sons de clique ao interagir com os elementos e adiciona um áudio de fundo que pode ser pausado ou reproduzido.
-

Descrição por Funções

1. **main()**
 - **Função Principal:** Inicia a aplicação.
 - **Tarefas executadas:**

1. Seleciona o elemento <canvas> do DOM onde as imagens serão renderizadas.
2. Cria uma instância da classe ISearchEngine apontando para o ficheiro database.json.
3. Expõe a instância da aplicação globalmente (variável window.app) para facilitar depurações durante o desenvolvimento.
4. Chama o método init(canvas) da classe ISearchEngine para inicializar a aplicação.
5. Adiciona um event listener para redimensionar o canvas dinamicamente sempre que a janela for redimensionada (resize).

2. updateCanvasSize()

- **Função Auxiliar:** Ajusta o tamanho do canvas para ocupar a largura da janela.
- Atualiza o atributo width do canvas com a largura do corpo (document.body.clientWidth).
- Re-renderiza as imagens no canvas chamando o método gridView da aplicação.

3. playClickSound()

- **Propósito:** Reproduz um som de clique cada vez que o utilizador interage com certos elementos da interface.
- Define o tempo de reprodução para o início do som (currentTime = 0) e chama o método play() no elemento <audio>.

4. Event Listeners para Interação

- **Botão de Pesquisa (button-search)**
 - Liga a função processar() que executa uma pesquisa de palavras-chave com base no valor do input de texto.
- **Cores Dominantes (.dominant-box)**
 - Quando clicado, chama o método searchKeywords da aplicação passando a cor como argumento.
- **Histograma de Cores (.color-box)**
 - Quando clicado, realiza uma pesquisa com base na cor selecionada e na categoria especificada pelo input de texto.
- **Botões de Rádio (.radio input[type="radio"])**

- Atualiza automaticamente o valor do input de texto com a categoria associada ao botão de rádio selecionado e realiza uma pesquisa por palavras-chave.

5. Integração com Áudio de Fundo

- Adiciona ou seleciona o elemento <audio> para tocar um arquivo de música.
- Garante que o áudio é reproduzido em loop e com volume reduzido.
- Adiciona um botão de "Play/Pause" para permitir que o utilizador controle a reprodução.

Resumo

Este ficheiro é o ponto central de coordenação da aplicação **ISearchEngine**. Ele integra:

1. A lógica do motor de busca implementada na classe **ISearchEngine**.
 2. Os elementos do DOM (canvas, botões, cores, e categorias).
 3. Funções auxiliares para garantir que a interface seja interativa e responsiva.
-

5. Interface

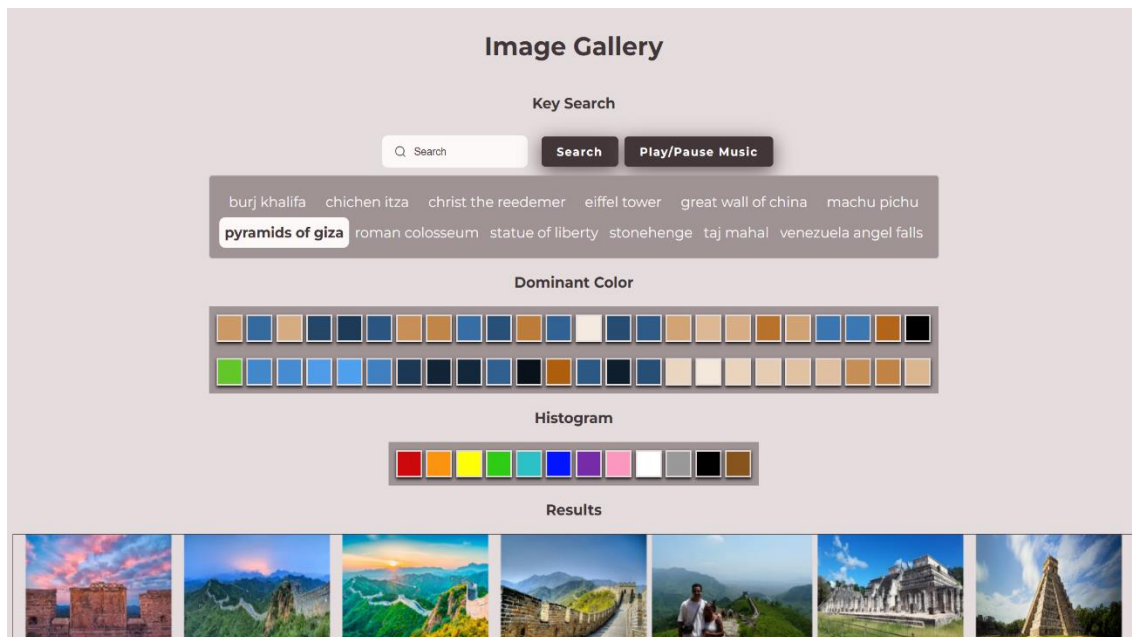


Figura 6: Aparência do projeto

Interface do Projeto

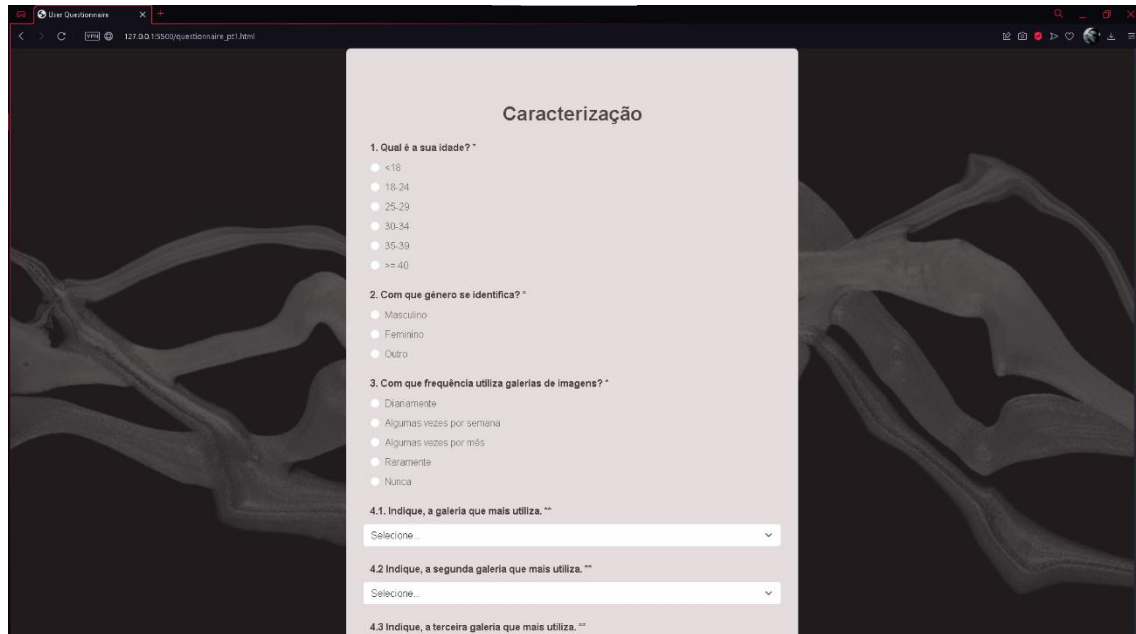
A interface gráfica do projeto foi desenhada para proporcionar uma interação intuitiva e eficaz para o utilizador. Para a sua implementação, utilizámos diversas ferramentas e componentes que permitem apresentar as funcionalidades principais do sistema de forma clara e organizada.

Escolhemos uma paleta de cores pastel para trazer um ambiente sereno e tranquilo ao utilizador, paleta esta que será também utilizada no questionário para manter a coerência e interação entre ambos.

6. Extra - Questionário

Decidimos criar um questionário de utilizador utilizando como base o realizado no TP3, assim, realizando algumas adaptações e aplicando a paleta base do trabalho podemos manter o utilizador dentro do projeto criando uma semelhança entre ambos.

Com as respostas do questionário podemos compreender e melhorar funções não tão bem recebidas pelos utilizadores.



The image shows a web browser window with a questionnaire titled "Caracterização". The questionnaire is displayed on a light pink background, which is centered on a dark background with abstract, wavy, light-colored patterns. The questionnaire consists of several questions with radio button or dropdown options.

Caracterização

1. Qual é a sua idade? *

- ☐ <18
- ☐ 18-24
- ☐ 25-29
- ☐ 30-34
- ☐ 35-39
- ☐ >= 40

2. Com que género se identifica? *

- ☐ Masculino
- ☐ Feminino
- ☐ Outro

3. Com que frequência utiliza galerias de imagens? *

- ☐ Diariamente
- ☐ Algumas vezes por semana
- ☐ Algumas vezes por mês
- ☐ Raramente
- ☐ Nunca

4.1. Indique, a galeria que mais utiliza. **

Selecione...

4.2 Indique, a segunda galeria que mais utiliza. **

Selecione...

4.3 Indique, a terceira galeria que mais utiliza. **

Figura 7: Aparência de uma página do questionário

7. Conclusões

O desenvolvimento da aplicação "Image Gallery" permitiu aplicar e consolidar os conhecimentos adquiridos ao longo da unidade curricular de Produção de Conteúdos Multimédia, explorando tecnologias como HTML5, CSS e JavaScript. Através da implementação de funcionalidades inovadoras e centradas na experiência do utilizador, o projeto demonstrou como conceitos multimédia podem ser utilizados para criar uma solução interativa e funcional.

Entre os principais destaques do projeto estão a pesquisa por palavras-chave e cores, a introdução de uma nuvem de palavras, integração de áudio, animações intuitivas na interface e o processamento eficiente de imagens para gerar histogramas. Os testes de usabilidade realizados confirmaram que a aplicação oferece uma experiência satisfatória e fácil de utilizar, evidenciando a eficácia das funcionalidades implementadas.

Por fim, o projeto alcançou os objetivos propostos demonstrando a importância da integração entre design, usabilidade e engenharia para criar produtos digitais inovadores.